



دانشگاه قم

دانشکده فنی و مهندسی

پروژه پایانی کارشناسی رشته مهندسی کامپیوتر

عنوان:

کاربرد احتمال در پیش بینی حملات سایبری و تحلیل رفتار ترافیک
شبکه

استاد راهنما:

دکتر خیرالله رهسپار فرد

نگارنده:

امید شعبانعلی

اسفند ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

تقدیر:

از زحمات استاد محترم راهنما و همه اساتید عزیز تشکر و قدردانی می نمایم

چکیده:

با افزایش حملات هکری، تشخیص به موقع فعالیت‌های مشکوک در شبکه اهمیت زیادی یافته است. روش‌های سنتی که تنها حملات شناخته شده را شناسایی می‌کنند، در برابر روش‌های جدید کارایی لازم را ندارند. در این پروژه، یک سیستم تشخیص نفوذ طراحی شده است که به جای لیست کردن حملات، «رفتار غیرعادی» شبکه را تشخیص می‌دهد. برای این کار از مدل ریاضی **رگرسیون خطی** استفاده شده است.

در روند انجام این پروژه، ابتدا ترافیک معمولی شبکه ضبط گردید تا الگوی رفتار سیستم در حالت عادی استخراج شود؛ سپس با استفاده از **قاعده ۳-سیگما (3-Sigma Rule)** و محاسبه انحراف معیار، یک «آستانه مجاز» برای تعداد بسته‌های شبکه (PPS) به دست آمد. برای تست سیستم، با ابزار Zenmap به شبکه حمله شد. نتایج نشان داد که با شروع حمله، ترافیک از خط رگرسیون فاصله گرفته و سیستم سریعاً این ناهنجاری را تشخیص می‌دهد. در نهایت، سیستم به گونه‌ای تنظیم شد که به محض تشخیص حمله، یک پیام هشدار خودکار به اپلیکیشن **ایتا** ارسال کند تا کاربر در لحظه مطلع شود. این پروژه نشان می‌دهد که با بهره‌گیری از ریاضیات و آمار و مفاهیم احتمال می‌توان یک سیستم امنیتی کاربردی و هوشمند ساخت.

واژه‌های کلیدی: امنیت شبکه، تشخیص نفوذ، رگرسیون خطی، ناهنجاری ترافیک، n8n، پیام‌رسان ایتا.

فهرست مطالب

فصل ۱: مقدمه و نمای کلی سیستم	۱
۱-۱- مقدمه و اهمیت مانیتورینگ شبکه	۲
۱-۲- تشریح مسئله: چرا تشخیص ناهنجاری (Anomaly Detection)?	۲
۱-۳- اهداف و سوالات اصلی پروژه	۳
۱-۳-۱- اهداف اصلی پروژه	۳
۱-۳-۲- سوالات کلیدی	۳
۱-۴- ضرورت و نوآوری پروژه	۳
۱-۵- معرفی ساختار گزارش	۴
فصل ۲: مفاهیم پایه و مبانی آماری پروژه	۵
۲-۱- واحد اندازه‌گیری ترافیک (PPS)	۶
۲-۲- رگرسیون خطی به زبان ساده	۶
۲-۳- تحلیل باقی‌مانده‌ها (Residuals)	۶
۲-۴- محاسبه آستانه بحرانی (The ۳-Sigma Rule)	۶
۲-۵- ابزار پویش Nmap و تاثیر آن بر آمار	۷
فصل ۳: معرفی ابزارها و زیرساخت اجرایی سیستم	۸
۳-۱- معرفی نرم‌افزار Wireshark	۹
۳-۲- کتابخانه Scapy در پایتون	۱۰
۳-۳- ابزار پویش شبکه Zenmap (Nmap)	۱۰
۳-۴- پلتفرم اتوماسیون n8n	۱۰
۳-۵- پیام‌رسان ایتا و API ایتایار	۱۱
فصل ۴: پیاده‌سازی فنی و تشریح سیستم	۱۲
۴-۱- مقدمه فنی	۱۳
۴-۲- پیش‌پردازش و آماده‌سازی داده‌ها (Preprocessing)	۱۳
۴-۳- پیاده‌سازی مدل رگرسیون خطی	۱۳
۴-۴- محاسبه آستانه تشخیص ناهنجاری (Threshold)	۱۴
۴-۵- مکانیزم مانیتورینگ زنده و شنود پکت‌ها	۱۵
۴-۶- اتصال به شبکه و اتوماسیون هشدار (Integration)	۱۶
۴-۷- مکانیزم پاسخ فعال و مسدودسازی خودکار آی‌پی (Active Response & IP Blocking)	۱۶
۴-۷-۱- معماری و روند عملکرد واکنش خودکار	۱۷

فصل ۵: آزمایش، ارزیابی و نتیجه‌گیری

۱۹

- ۵-۱- مقدمه ۲۰
- ۵-۲- سناریوی آزمایش ۲۰
- ۵-۳- مشاهده خروجی در حالت عادی ۲۰
- ۵-۴- اجرای حمله و واکنش سیستم ۲۰
- ۵-۵- دریافت هشدار در پیام‌رسان ایتا ۲۲
- ۵-۶- نتیجه‌گیری نهایی ۲۲
- ۵-۷- پیشنهادات برای آینده ۲۲

منابع

۲۳

- منابع ۲۴

پیوست‌ها

۲۵

فصل ۱:

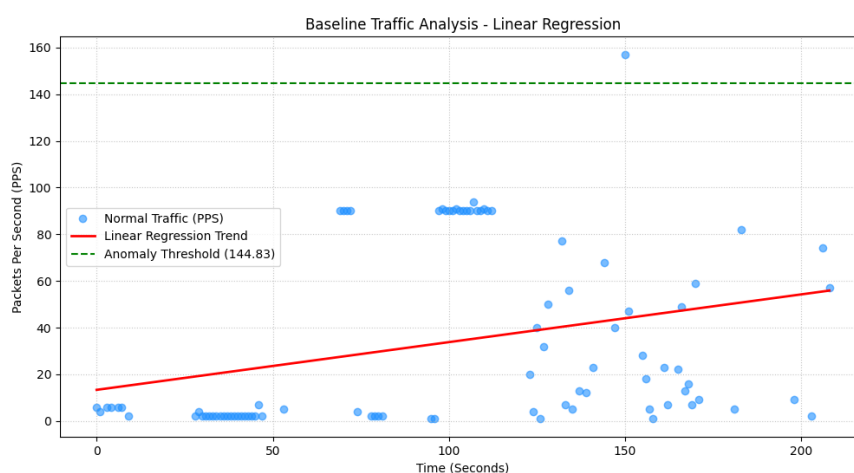
مقدمه و نمای کلی سیستم

۱-۱- مقدمه و اهمیت مانیتورینگ شبکه

امروزه شبکه‌های کامپیوتری ستون فقرات هر مجموعه‌ای هستند و پایداری آن‌ها مستقیماً بر عملکرد سیستم‌ها تاثیر می‌گذارد. با پیچیده‌تر شدن ابزارهای نفوذ، دیگر نمی‌توان تنها به فایروال‌های ساده تکیه کرد. مانیتورینگ یا پایش مداوم ترافیک، یکی از کلیدی‌ترین کارها برای حفظ امنیت است. اگر ما بدانیم در هر لحظه چه اتفاقی در شبکه می‌افتد، می‌توانیم قبل از اینکه یک نفوذ به فاجعه تبدیل شود، جلوی آن را بگیریم. در واقع امنیت شبکه و روش‌های مقابله با نفوذگران، نیازمند یک نگاه دقیق به آمارهای لحظه‌ای شبکه است [۱]

۱-۲- تشریح مسئله : چرا تشخیص نا هنجاری (Anomaly Detection)?

اکثر سیستم‌های امنیتی قدیمی (مثل آنتی‌ویروس‌ها) بر اساس "امضا" کار می‌کنند؛ یعنی فقط حملاتی را می‌شناسند که قبلاً تعریف شده باشند. اما مشکل اینجاست که هکرها مدام از روش‌های جدید استفاده می‌کنند. در این پروژه، مسئله اصلی این است که ما به جای گشتن دنبال یک حمله خاص، به دنبال "رفتار غیرعادی" هستیم. ترافیک شبکه در حالت عادی یک نظم مشخص دارد، اما وقتی حمله‌ای مثل اسکن پورت‌ها (Port Scanning) اتفاق می‌افتد، این نظم به هم می‌خورد. هدف ما این است که با استفاده از مدل ریاضی رگرسیون خطی، این بی‌نظمی را شناسایی کنیم [۲]



عکس ۱-۱: تفاوت ترافیک منظم شبکه در حالت عادی و جهش ناگهانی پکت‌ها در زمان حمله

۳-۱- اهداف و سوالات اصلی پروژه

۱-۳-۱- اهداف اصلی پروژه

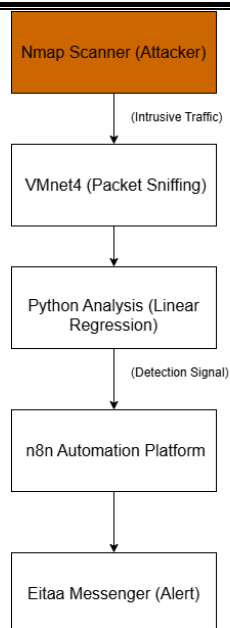
- **ثبت رفتار عادی (Baseline):** بررسی ترافیک واقعی شبکه (روی اینترفیس VMnet4) برای فهمیدن اینکه در حالت معمولی چند پکت در ثانیه جابجا می‌شود.
- **محاسبه آستانه مجاز (Threshold):** پیدا کردن یک عدد مشخص به عنوان "مرز خطر" با استفاده از فرمول‌های ریاضی، به طوری که نوسانات کوچک شبکه باعث پیام اشتباه نشود.
- **اتوماسیون اطلاع‌رسانی:** ایجاد یک سیستم هوشمند که به محض دیدن ترافیک غیرعادی، خودش پیام هشدار را به اپلیکیشن ایتا بفرستد.

۱-۳-۲- سوالات کلیدی

۱. آیا یک فرمول ریاضی ساده (رگرسیون) می‌تواند جایگزین سیستم‌های سنگین و گران‌قیمت امنیتی برای تشخیص اسکن‌های شبکه شود؟
۲. چطور می‌توان ترافیک‌های زمینه‌ای ویندوز (مثل SSDP) را از حملات واقعی تفکیک کرد؟
۳. سرعت پاسخگویی سیستم در زمان استفاده از ابزارهای اتوماسیون (n8n) چقدر است؟

۴-۱- ضرورت و نوآوری پروژه

بسیاری از سیستم‌های هوش مصنوعی فعلی برای اجرا نیاز به سخت‌افزارهای قوی و سنگین دارند. نوآوری این پروژه در "سبک بودن" آن است [۳]. ما از مدلی استفاده کردیم که با کمترین فشار به CPU، بیشترین دقت را در تشخیص اسکن‌های پرسرعت دارد. همچنین، استفاده از پلتفرم n8n برای ارسال پیام به ایتا، این پروژه را به یک ابزار کاملاً کاربردی و مدرن تبدیل کرده است.



عکس ۱-۲: نمودار کلی گردش کار سیستم (از ضبط پکت تا دریافت پیام در موبایل)

۱-۵- معرفی ساختار گزارش

این گزارش برای درک بهتر مراحل کار، به ۵ بخش تقسیم شده است:

- **فصل اول:** همین مقدمه و اهدافی که بررسی کردیم.
- **فصل دوم:** بررسی مبانی آماری و فرمول‌هایی که برای تحلیل ترافیک استفاده کردیم.
- **فصل سوم:** معرفی نرم‌افزارها و ابزارهایی مثل Scapy ، Nmap و n8n.
- **فصل چهارم:** جزئیات پیاده‌سازی، کدهای پایتون و نحوه تنظیم وب‌هوک.
- **فصل پنجم:** تست عملی سیستم، مشاهده نتایج در ایتا و نتیجه‌گیری نهایی.

فصل ۲:

مفاهیم پایه و مبانی آماری پروژه

۲-۱- واحد اندازه‌گیری ترافیک (PPS)

در این پروژه، به جای تمرکز بر حجم داده‌ها (بایت)، از واحد PPS یا Packet Per Second استفاده کردیم. علت این انتخاب این است که در حملات پویش شبکه (Scanning)، مهاجم تعداد زیادی پکت کوچک برای بررسی پورت‌ها می‌فرستد. بنابراین، شمارش تعداد بسته‌ها در هر ثانیه، معیار بسیار دقیق‌تری برای تشخیص ناهنجاری نسبت به حجم ترافیک است [۴].

۲-۲- رگرسیون خطی به زبان ساده

رگرسیون خطی یکی از ساده‌ترین و در عین حال کاربردی‌ترین روش‌های آماری است. هدف این مدل، پیدا کردن رابطه‌ای بین زمان (محور افقی یا X) و تعداد بسته‌ها (محور عمودی یا Y) است. در واقع این مدل سعی می‌کند خطی را رسم کند که از نزدیک‌ترین فاصله ممکن نسبت به تمام نقاط ترافیک عادی عبور کند. معادله این خط به صورت $y = mx + b$ تعریف می‌شود که در آن m نشان‌دهنده شیب خط و b نشان‌دهنده عرض از مبدأ است.

۲-۳- تحلیل باقی‌مانده‌ها (Residuals)

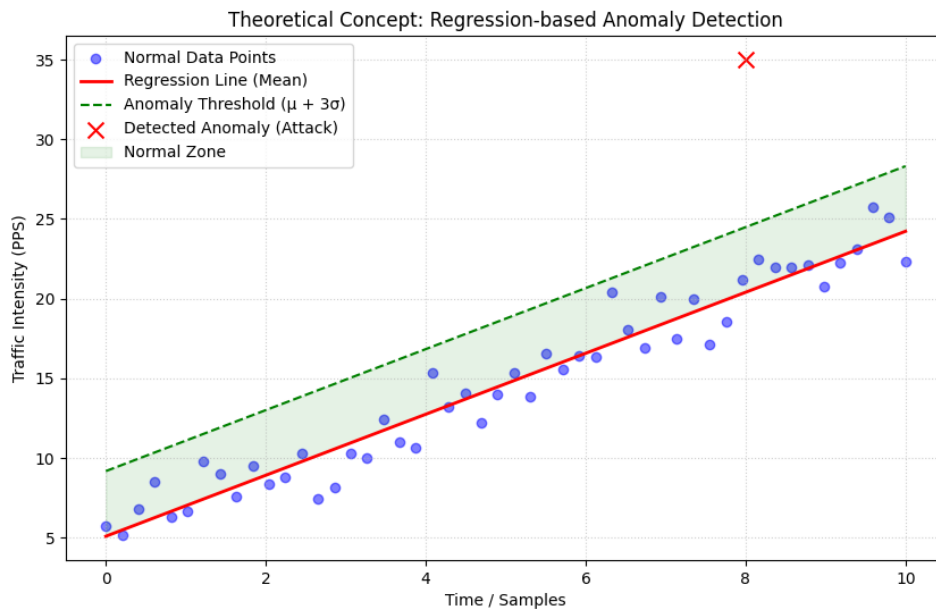
باقی‌مانده یا خطا، در واقع فاصله هر نقطه واقعی ترافیک تا خط رگرسیون است. در یک شبکه پایدار، این فاصله‌ها باید کم و ناچیز باشند. اما زمانی که حمله اتفاق می‌افتد، نقاط جدید ترافیک فاصله بسیار زیادی از خط پیدا می‌کنند. سیستم ما با محاسبه این فاصله‌ها متوجه می‌شود که رفتار شبکه از حالت پیش‌بینی شده خارج شده است [۵].

۲-۴- محاسبه آستانه بحرانی (The ۳-Sigma Rule)

یکی از بخش‌های مهم پروژه ما، تعیین عددی است که بالاتر از آن را به عنوان "حمله" در نظر بگیریم. برای این کار از روش خطای استاندارد (Standard Error) استفاده کردیم. طبق کدی که پیاده‌سازی شده، مراحل زیر طی شد: ابتدا میانگین ترافیک (Average PPS) به دست آمد. سپس میزان انحراف استاندارد خطا یا همان Se محاسبه شد. در نهایت، آستانه نهایی از فرمول زیر به دست آمد:

$$Threshold = Average + (3 * SE)$$

مبنای نظری انتخاب ضریب ۳ در این رابطه، **قانون ۳-سیگما** در توزیع نرمال است [۶]. طبق این اصل آماری، ۹۹.۷٪ از نوسانات ترافیک عادی شبکه درون محدوده $Average \pm 3SE$ قرار می‌گیرند [۷]. بنابراین، عبور ترافیک از این مرز با اطمینان آماری بالا (حدود ۹۹.۷٪) نشان‌دهنده وقوع ناهنجاری یا حمله در شبکه است



عکس ۲-۱: نمایش هندسی قانون ۳-سیگما و نحوه تعیین آستانه ناهنجاری در اطراف خط رگرسیون.

۲-۵- ابزار پویش Nmap و تاثیر آن بر آمار

ابزار Nmap در حالت‌های تهاجمی (مثل سویچ T^4) بسته‌ها را با سرعت بسیار بالا و بدون وقفه در شبکه می‌فرستد. این کار باعث می‌شود مقدار ترافیک در مدل رگرسیون ما به ناگهان جهش کند و از آستانه یا همان Threshold که در بخش قبل حساب کردیم، عبور کند. سیستم در این لحظه با مقایسه ترافیک لحظه‌ای و آستانه تعیین شده، وضعیت حمله را شناسایی کرده و هشدار صادر می‌کند.

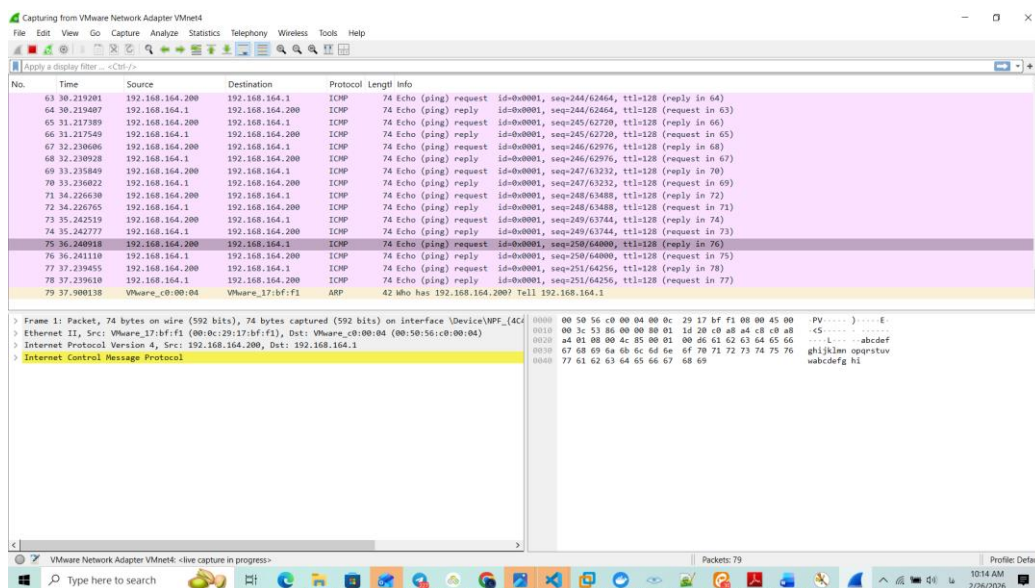
فصل ۳:

معرفی ابزارها و زیرساخت اجرایی

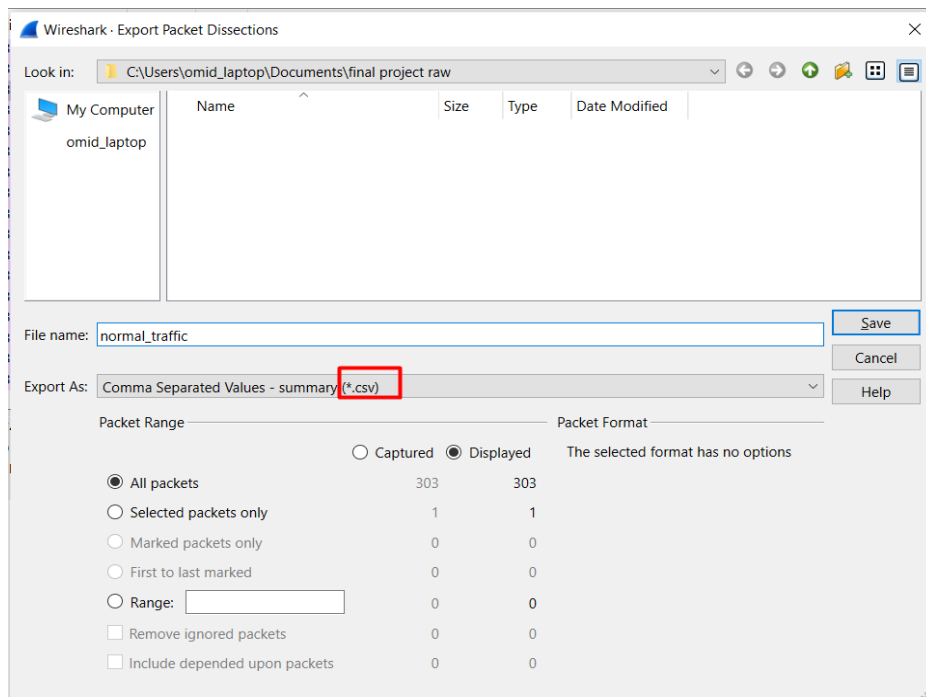
سیستم

۱-۳- معرفی نرم افزار Wireshark

ما در این پروژه برای جمع آوری داده های اولیه (Baseline) از وایر شارک استفاده کردیم [۸]. این ابزار به ما اجازه داد تا بفهمیم در شبکه مجازی ما (VMnet4) چه پکت هایی در حال رد و بدل شدن است. خروجی این ابزار به صورت فایل CSV استخراج شد تا برای پایتون قابل فهم باشد.



عکس ۱-۳: محیط نرم افزار Wireshark در حال پایش و ثبت ترافیک اولیه (Baseline) شبکه در اینترنت فیس ۴ VMnet



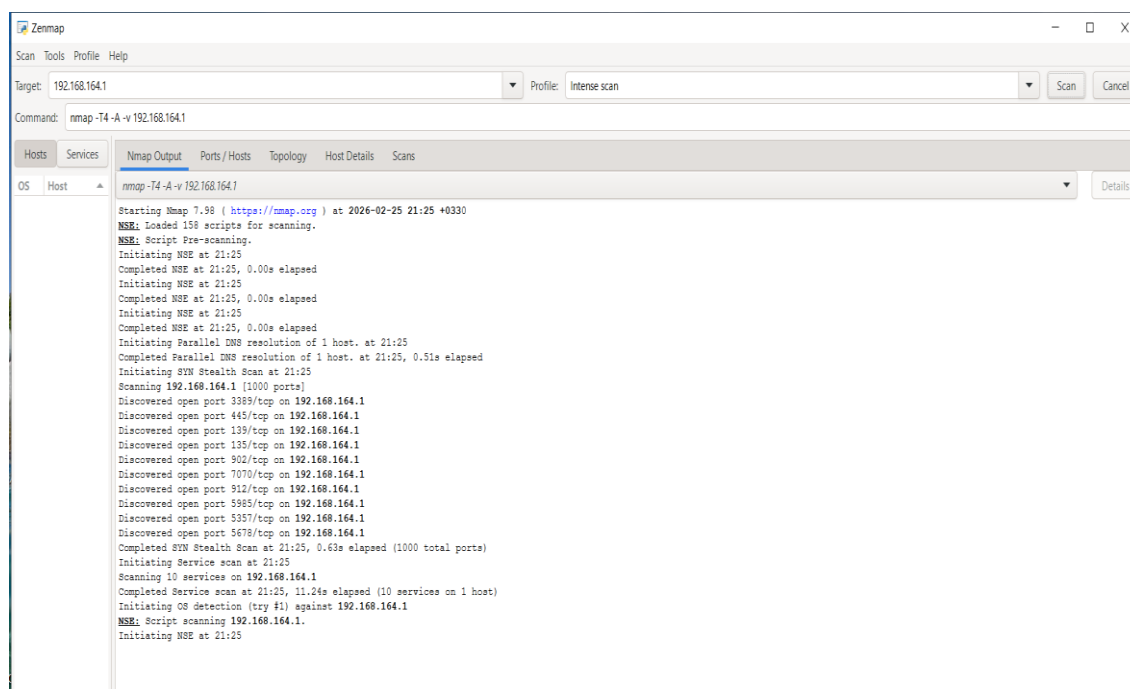
عکس ۲-۳: فرآیند شنود داده های شبکه و آماده سازی فایل خروجی CSV برای پردازش در مدل رگرسیون

۳-۲- کتابخانه Scapy در پایتون

قلب بخش مانیتورینگ زنده ما، کتابخانه Scapy است. برخلاف کتابخانه‌های معمولی، Scapy به ما اجازه می‌دهد مستقیماً به کارت شبکه وصل شویم و پکت‌ها را در لحظه "شنود (Sniff)" کنیم [۹]. ما از این ابزار برای شمارش پکت‌ها در بازه‌های ۱ ثانیه‌ای استفاده کردیم.

۳-۳- ابزار پویش شبکه (Nmap) Zenmap

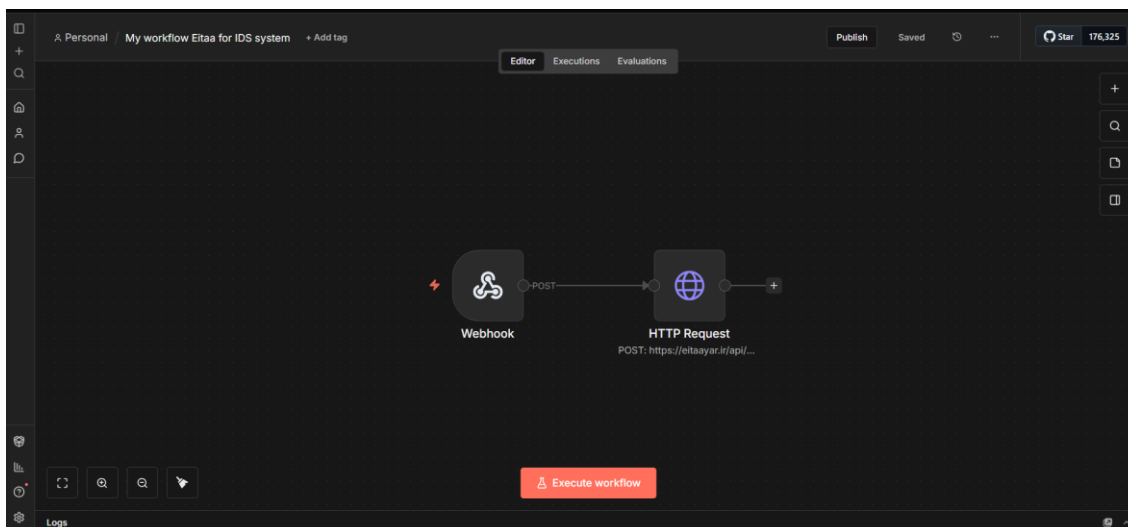
برای اینکه سیستم را تست کنیم، نیاز به یک "مهاجم" داشتیم Zenmap. نسخه گرافیکی Nmap است که با تنظیم آن روی حالت **Intense Scan**، ترافیک سنگینی ایجاد کردیم تا ببینیم آیا مدل رگرسیون ما متوجه این تغییر ناگهانی می‌شود یا خیر.



عکس ۳-۳: محیط نرم‌افزار Zenmap در حال اجرای اسکن روی آی‌پی هدف

۳-۴- پلتفرم اتوماسیون n8n

یکی از بخش‌های مدرن این پروژه استفاده از n8n است [۱۰]. این ابزار به عنوان یک "واسط" عمل می‌کند. وقتی پایتون حمله را تشخیص می‌دهد، یک سیگنال به n8n می‌فرستد و n8n وظیفه دارد این سیگنال را به پیام قابل فهم برای "بات ایتا" تبدیل کند. این کار باعث می‌شود سیستم ما بدون نیاز به مانیتور کردن مداوم کنسول پایتون، هشدارها را به موبایل ما بفرستد.



عکس ۳-۴: نمای محیط کاربری n8n و گره‌های (Nodes) متصل شده به هم

۳-۵- پیام‌رسان ایتا و API ایتایار

برای بخش اطلاع‌رسانی، از پیام‌رسان بومی ایتا استفاده شد. با استفاده از قابلیت Webhook و اتصال آن به n8n، توانستیم یک کانال ارتباطی امن برای دریافت هشدارهای لحظه‌ای ایجاد کنیم.

فصل ۴:

پیاده‌سازی فنی و تشریح سیستم

۴-۱- مقدمه فنی

در این فصل به بررسی جزئیات پیاده‌سازی کدها، نحوه پردازش داده‌های خام و اتصال بخش‌های مختلف سیستم به یکدیگر می‌پردازیم. پیاده‌سازی این پروژه شامل سه بخش اصلی است: استخراج ویژگی‌ها از ترافیک، مدل‌سازی ریاضی و در نهایت مانیتورینگ زنده.

۴-۲- پیش‌پردازش و آماده‌سازی داده‌ها (Preprocessing)

داده‌های خام که از نرم‌افزار وایرشارک به صورت فایل CSV استخراج شده بودند، توسط کتابخانه pandas در پایتون فراخوانی شدند. از آنجایی که ترافیک شبکه در لحظات مختلف نوسان دارد، ما داده‌ها را بر اساس ستون زمان (Time) گروه‌بندی کردیم تا تعداد بسته‌ها در هر ثانیه یا همان PPS مشخص شود. در این مرحله، نوسانات ناگهانی ترافیک (Burst Traffic) نیز به عنوان بخشی از رفتار طبیعی شبکه در داده‌های آموزشی حفظ شدند تا مدل نسبت به فعالیت‌های عادی پرحجم حساسیت کاذب نشان نداده و آستانه تشخیص (Threshold) با دقت بیشتری تعیین گردد [۱۱]. این کار به ما کمک کرد تا یک دید کلی از وضعیت ترافیک نرمال شبکه به دست آوریم.

```
# Preprocessing: Round time to nearest second
# This allows us to count packets per second (PPS)
df['Time'] = df['Time'].astype(float).round(0)

# Group by 'Time' to get the count of packets in each second
pps_data = df.groupby('Time').size().reset_index(name='Packet_Count')
```

شرح برنامه ۴-۱: در این قطعه کد، از کتابخانه pandas برای گروه‌بندی داده‌های خام و استخراج نرخ پکت در ثانیه استفاده شده است.

۴-۳- پیاده‌سازی مدل رگرسیون خطی

برای مدل‌سازی ترافیک، از کلاس LinearRegression در کتابخانه scikit-learn استفاده شد. مدل سعی می‌کند رابطه‌ای بین زمان (محور x) و تعداد بسته‌ها (محور y) پیدا کند. این رابطه در ساده‌ترین حالت به صورت فرمول $y = wx + b$ تعریف می‌شود. پس از آموزش دیدن مدل روی داده‌های عادی، "باقی‌مانده‌ها" یا همان Residuals محاسبه شدند. باقی‌مانده در واقع تفاضل بین مقدار واقعی مشاهده شده در شبکه و مقداری است که مدل ما پیش‌بینی

کرده است. فرمول آن به زبان ساده به این صورت است:

مقدار پیش‌بینی شده - مقدار واقعی = باقی‌مانده

مدل رگرسیون مورد استفاده، با کمترین میزان خطای میانگین مربعات (MSE) بر روی داده‌های آموزشی برازش شد تا پیش‌بینی ترافیک نرمال با دقت حداکثری انجام پذیرد و انحرافات جزئی نیز قابل شناسایی باشند.

```
# Initialize and fit the Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Predict values to calculate residuals
y_pred = model.predict(X)
```

شرح برنامه ۴-۲: برازش مدل رگرسیون خطی بر روی داده‌های آموزشی جهت استخراج خط روند ترافیک نرمال.

۴-۴- محاسبه آستانه تشخیص ناهنجاری (Threshold)

مهم‌ترین بخش آماری این پروژه، محاسبه "خطای استاندارد برآورد" یا همان Se است. این عدد به ما می‌گوید که داده‌های عادی شبکه چقدر از خط اصلی رگرسیون فاصله دارند. برای تعیین آستانه نهایی و مرز تشخیص حمله، از میانگین ترافیک به علاوه ۳ برابر خطای استاندارد استفاده کردیم. این کار باعث می‌شود دقت سیستم بالا برود و نوسانات معمولی شبکه به اشتباه "حمله" در نظر گرفته نشوند (کاهش هشدارهای اشتباه). فرمول نهایی ما به این صورت بود:

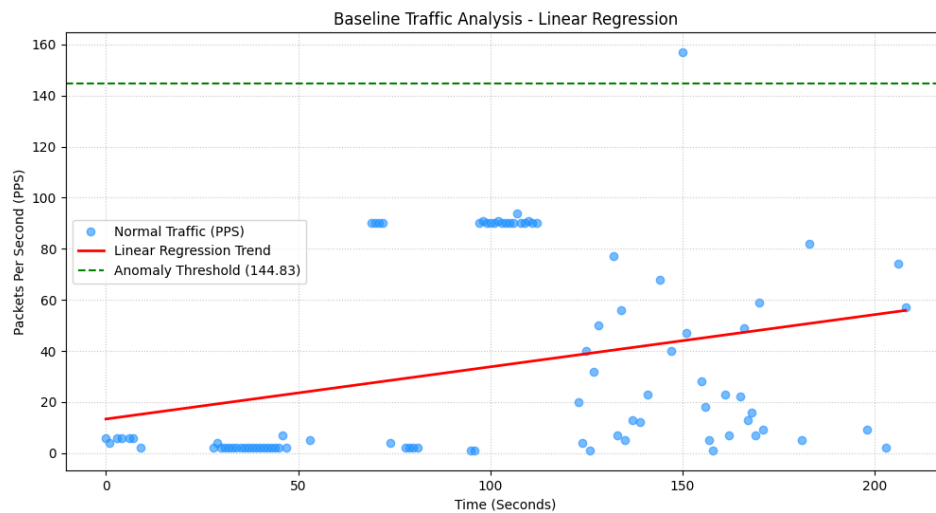
$$\text{Threshold} = \text{Average} + (3 * \text{Se})$$

```
# Calculate Standard Error of the Estimate (Se)
# Se = sqrt( sum( (y_actual - y_pred)^2 ) / (n - 2) )
residuals = y - y_pred
se = np.sqrt(np.sum(residuals**2) / (len(y) - 2))

# Calculate Threshold (3-sigma rule: 3 * Se above the regression line)
# We use this as our anomaly detection boundary
avg_pps = np.mean(y)
threshold = avg_pps + (3 * se)
```

شرح برنامه ۴-۳: محاسبه خطای استاندارد برآورد و تعیین آستانه نهایی تشخیص حمله با استفاده از قانون ۳-سیگما.

طبق محاسبات انجام شده روی داده‌های واقعی این پروژه، عدد ۱۴۴.۸۳ به عنوان مرز بحرانی یا آستانه نهایی به دست آمد.



شکل ۴-۱: نمودار تحلیل رگرسیون ترافیک واقعی و تعیین آستانه بحرانی (Threshold) بر اساس داده‌های شبکه.

توضیحات شکل بدر این نمودار، نقاط آبی نشان‌دهنده تعداد بسته‌ها در ثانیه (PPS) در طول زمان هستند. خط قرمز، روند کلی ترافیک نرمال را مدل‌سازی کرده و خط چین سبز، آستانه بحرانی (۱۴۴.۸۳) را نشان می‌دهد که با فرمول $3\sigma_e + Average$ به دست آمده است. عبور نقطه ناهنجار از مرز مشخص شده، لحظه دقیق شناسایی حمله Nmap را تایید می‌کند.

۵-۴- مکانیزم مانیتورینگ زنده و شنود پکت‌ها

در این بخش از کتابخانه قدرتمند Scapy استفاده شده است. اسکریپت اصلی پروژه با استفاده از تابع sniff روی کارت شبکه مجازی، بسته‌ها را در بازه‌های یک ثانیه‌ای می‌شمارد. منطق تشخیص حمله در کد بسیار ساده و سریع طراحی شده است؛ به این صورت که اگر تعداد بسته‌های شمارش شده در یک ثانیه (PPS لحظه‌ای) از عدد آستانه (۱۴۴.۸۳) بیشتر شود، سیستم وضعیت را "بحرانی" تشخیص داده و بلافاصله تابع ارسال هشدار را اجرا می‌کند.

```
while True:
    packet_count = 0
    ip_counter.clear()

    # Sniff packets for 1 second
    scapy.sniff(iface=INTERFACE, prn=process_packet, timeout=1, store=0)

    current_pps = packet_count
    print(f"Current Traffic: {current_pps} PPS", end='\n')

    if current_pps > THRESHOLD:
        # Capture accurate system time upon attack detection
        attack_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        print(f"\n[!!!] ANOMALY DETECTED at [{attack_time}]! Traffic: {current_pps} PPS")

        attacker_ip = "Unknown"
        if ip_counter:
            attacker_ip = ip_counter.most_common(1)[0][0]
            print(f"[!] Attacker IP identified: {attacker_ip}")

        # Active response execution
        block_ip(attacker_ip)

        send_alert(current_pps, attacker_ip)
        time.sleep(5)
```

شرح برنامه ۴-۴: مانیتورینگ بلادرنگ اینترنتی شبکه و مقایسه PPS لحظه‌ای با آستانه بحرانی جهت تشخیص ناهنجاری.

```
Administrator: C:\Windows\system32\cmd.exe - python detect_attack.py
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd C:\Drive D Backup 1405-03-5\Omid\0000 00000000\000 7\Final Project\project\src
C:\Drive D Backup 1405-03-5\Omid\0000 00000000\000 7\Final Project\project\src>python detect_attack.py
[*] Monitoring started on VMware Network Adapter VMnet4...
[*] Threshold is set to: 144.83 PPS
Current Traffic: 184 PPS
[!!!] ANOMALY DETECTED at [2026-07-28 20:11:06]! Traffic: 184 PPS
[!] Attacker IP identified: 192.168.164.200
[+] Successfully blocked ALL traffic for IP 192.168.164.200 in Windows Firewall.
[+] Alert sent to n8n! Traffic: 184 PPS | Target IP: 192.168.164.200
```

عکس ۴-۲: اسکرین‌شات از محیط خط فرمان (cmd) در لحظه شناسایی ناهنجاری و صدور هشدار سیستمی.

۶-۴- اتصال به شبکه و اتوماسیون هشدار (Integration)

زمانی که حمله تشخیص داده می‌شود، سیستم یک درخواست به سمت سرور n8n ارسال می‌کند. این داده‌ها شامل اطلاعاتی مثل زمان دقیق حمله و تعداد پکت‌ها است. پلتفرم n8n با دریافت این اطلاعات، بلافاصله از طریق یک رابط (Webhook) پیام را به بات طراحی شده در پیام‌رسان ایتا منتقل می‌کند. این زنجیره باعث می‌شود که مدیر شبکه بدون نیاز به چک کردن مداوم سیستم، از طریق موبایل خود در جریان حملات قرار بگیرد.

```
def send_alert(pps, attacker_ip):
    payload = {
        "method": "Linear Regression Model",
        "traffic": pps,
        "threshold": THRESHOLD,
        "status": "Attack Detected & Blocked",
        "message": "Intrusion detected by Linear Regression Model",
        "student": "Omid",
        "network": "VMware VMnet4",
        "blocked_ip": attacker_ip,
        "action": "Active Response - Firewall Block"
    }
```

شرح برنامه ۴-۵: متد ارسال داده‌های ناهنجاری به وب‌هوک n8n جهت اطلاع‌رسانی خودکار در پیام‌رسان ایتا

۷-۴- مکانیزم پاسخ فعال و مسدودسازی خودکار آی‌پی (Active

(Response & IP Blocking)

در سامانه‌های تشخیص نفوذ سنتی (Passive IDS)، سیستم تنها پس از شناسایی آنومالی اقدام به ثبت گزارش و ارسال هشدار به مدیر شبکه می‌نماید. اما در سناریوهای حملات منع خدمت (DoS/DDoS)

یا اکسپلویت‌های سریع شبکه، فاصله زمانی میان اعلان هشدار تا اقدام دستی مدیر شبکه می‌تواند منجر به آسیب‌پذیری شدید یا از دسترس خارج شدن سرویس‌ها گردد. جهت رفع این چالش، سامانه طراحی شده در این پروژه به ماژول پاسخ فعال (Active Response) مجهز گردید تا عملیات پیشگیری از نفوذ (IPS) به صورت خودکار انجام پذیرد. [۱۲]

۱-۷-۴- معماری و روند عملکرد واکنش خودکار

فرآیند شناسایی تا مسدودسازی خودکار آی‌پی مهاجم طی ۶ گام عملیاتی به شرح زیر انجام می‌پذیرد:

۱. **تحلیل عمیق بسته‌ها (Packet Inspection):** تابع `process_packet` حین شنود ترافیک

کارت شبکه (VMware VMnet^۴)، تمامی بسته‌های لایه شبکه را بررسی کرده و آدرس آی‌پی مبدأ (`src_ip`) را استخراج می‌نماید.

۲. **شمارش بر اساس آدرس مبدأ:** با به‌کارگیری ساختار داده Counter از کتابخانه `collections` در پایتون، تعداد بسته‌های ارسالی توسط هر آدرس IP به صورت مجزا در بازه‌های زمانی یک‌ثانیه‌ای ثبت می‌شود.

۳. **ارزیابی حد آستانه (Threshold Evaluation):** چنانچه مجموع بسته‌های دریافتی در یک ثانیه از حد آستانه مدل رگرسیون خطی (معادل ۱۴۴.۸۳ PPS) فراتر رود، وضعیت ناهنجاری صادر می‌شود.

۴. **شناسایی منبع اصلی حمله:** الگوریتم با فراخوانی متد `most_common(1)` آدرس آی‌پی که بیشترین سهم را در تولید ترافیک غیرمجاز در آن ثانیه داشته به عنوان آی‌پی مهاجم تفکیک می‌کند.

۵. **تزریق قانون به فایروال (Firewall Rule Injection):** ماژول `block_ip` بر اساس نوع سیستم‌عامل میزبان، دستور بومی فایروال را صادر می‌کند [۱۳]:

در محیط ویندوز:

اجرای دستور زیر در سطح دسترسی Administrator جهت مسدودسازی ترافیک ورودی:

```
netsh advfirewall firewall add rule name="IDS_AUTO_BLOCK_[IP]" dir=in  
action=block remoteip=[IP]
```

در محیط لینوکس:

اجرای دستور زیر در جدول `iptables`:

```
iptables -A INPUT -s [IP] -j DROP
```

۶. مدیریت وضعیت (State Management) و اعلام هشدار: جهت جلوگیری از کندی

فایروال ناشی از ثبت قوانین تکراری، آدرس‌های مسدودشده در مجموعه‌ای موقت (blocked_ips) ذخیره می‌گردند. در نهایت، داده‌های تحلیل به همراه آدرس IP مسدودشده در قالب یک محموله JSON به وب‌هوک n8n ارسال می‌شود تا گزارش آن به پیام‌رسان منتقل گردد.

فصل ۵:

آزمایش، ارزیابی و نتیجه‌گیری

۱-۵- مقدمه

در این فصل، کارایی سیستم طراحی شده را در مواجهه با شرایط واقعی بررسی می‌کنیم. هدف این است که ببینیم آیا مدل رگرسیونی که در فصل‌های قبل توضیح دادیم، واقعاً می‌تواند ترافیک عادی را از حمله تشخیص دهد و آیا فرآیند اطلاع‌رسانی به درستی انجام می‌شود یا خیر.

۲-۵- سناریوی آزمایش

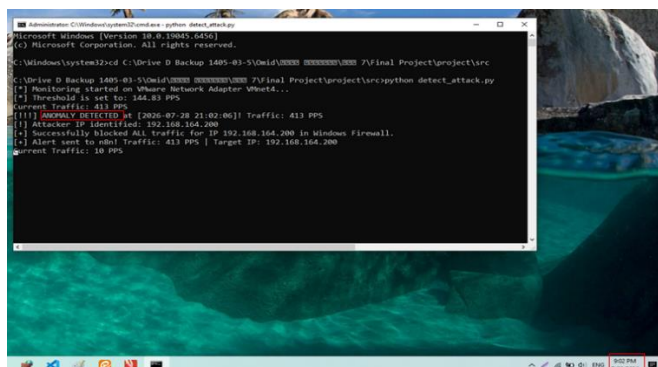
- برای تست سیستم، یک محیط آزمایشگاهی مجازی شامل دو بخش زیر ایجاد شد :
۱. **سیستم هدف (Target):** یک ماشین مجازی که کارت شبکه آن (VMnet۴) توسط اسکریپت پایتون ما مانیتور می‌شود.
 ۲. **سیستم مهاجم (Attacker):** استفاده از ابزار Zenmap برای ارسال پکت‌های پویش شبکه (Intense Scan) به سمت سیستم هدف.

۳-۵- مشاهده خروجی در حالت عادی

در ابتدای کار، سیستم در شرایطی قرار گرفت که فقط فعالیت‌های معمولی شبکه (مثل پکت‌های شناسایی و پندوز) در جریان بود. در این حالت، تعداد بسته‌ها در هر ثانیه (PPS) بسیار پایین‌تر از آستانه ۱۴۴.۸ بود. نمودار خروجی پایتون نشان داد که تمام نقاط ترافیکی در اطراف خط رگرسیون قرار دارند و هیچ هشدار اشتباهی صادر نشد.

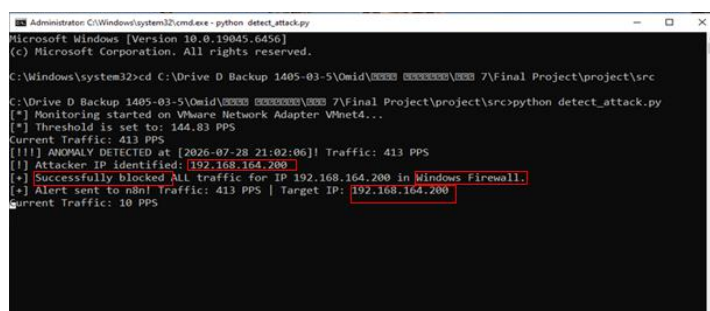
۵-۴- اجرای حمله و واکنش سیستم

- به محض شروع اسکن توسط Zenmap، تعداد بسته‌های ارسالی در ثانیه به شدت افزایش یافت و به عددی بالای ۶۰۰ پکت در ثانیه رسید. در این لحظه:
۱. سیستم پایتون بلافاصله جهش ترافیکی را تشخیص داد (چون مقدار PPS از آستانه ۱۴۴.۸۳ عبور کرد).
 ۲. در کنسول پایتون پیام "ANOMALY DETECTED" به همراه مقدار دقیق ترافیک چاپ شد.



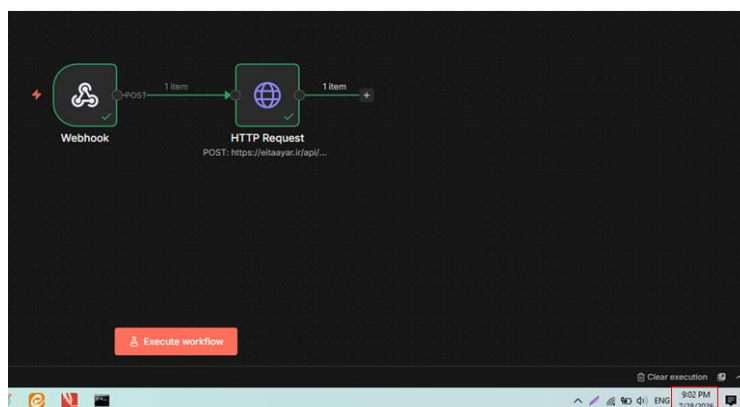
شکل ۵-۱: تشخیص ناهنجاری در محیط خط فرمان پایتون بلافاصله پس از شروع حمله.

۳. **اعمال پاسخ فعال (Active Response):** ماژول مسدودسازی بلافاصله وارد عمل شده و دستور افزودن قانون بلاک (Block Rule) را بر اساس آدرس IP مهاجم به فایروال سیستم عامل صادر نمود. با این اقدام، ترافیک مخرب در همان ثانیه ۱۰ PPS شد که تقریباً صفر است و در محدوده ترافیک نرمال قرار دارد.



شکل ۵-۲: تشخیص ناهنجاری و مسدودسازی خودکار IP در محیط خط فرمان پایتون

۴. درخواست وب هوک به سمت n8n ارسال گردید. با دریافت این درخواست، گره‌های (Nodes) مربوطه در جریان کاری فعال شدند.



شکل ۵-۳: وضعیت فعال جریان کاری (Workflow) در پلتفرم n8n در لحظه پردازش هشدار.

۵-۵- دریافت هشدار در پیام‌رسان ایتا

پس از تشخیص حمله، پلتفرم nAn طبق جریان کاری (Workflow) تعریف شده، اطلاعات را پردازش کرده و یک پیام شامل زمان حمله و شدت آن را به بات اختصاصی در پیام‌رسان ایتا ارسال کرد. تصویر زیر نمونه‌ای از هشدار دریافت شده روی تلفن همراه را نشان می‌دهد که سرعت بالای پاسخگویی سیستم (کمتر از ۲ ثانیه) را تایید می‌کند.



شکل ۵-۴: تاییدیه نهایی و دریافت هشدار هوشمند بر روی اپلیکیشن ایتا.

۵-۶- نتیجه‌گیری نهایی

این پروژه نشان داد که می‌توان با استفاده از ابزارهای ساده ریاضی مانند رگرسیون خطی، یک سیستم تشخیص نفوذ (IDS) سبک و کارآمد ساخت. مزیت اصلی این روش، بی‌نیاز بودن از سخت‌افزارهای گران‌قیمت و توانایی تشخیص حملات جدید بدون داشتن الگوی قبلی است. همچنین ترکیب پایتون با ابزارهای اتوماسیون مدرن مثل nAn، امکان مانیتورینگ از راه دور را برای مدیران شبکه به راحتی فراهم می‌کند.

۵-۷- پیشنهادات برای آینده

- برای ارتقای این پروژه در آینده می‌توان موارد زیر را مد نظر قرار داد:
- استفاده از مدل‌های رگرسیون پیچیده‌تر برای شبکه‌هایی که نوسانات بسیار زیادی دارند.
 - گسترش سیستم برای تشخیص انواع دیگر حملات مانند حملات منع سرویس (DoS).
 - ایجاد یک داشبورد گرافیکی با استفاده از Grafana برای مشاهده لحظه‌ای نمودار رگرسیون و ترافیک شبکه

منابع

- [۱] ا. ملکيان, نفوذگري در شبكه و روش‌هاي مقابله, تهران: انتشارات نص, ۱۴۰۲.
- [۲] G. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning, Insecure.Com LLC, ۲۰۰۹.
- [۳] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. ۷, pp. ۴۱۵۲۵-۴۱۵۵۰, ۲۰۱۹.
- [۴] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," National Institute of Standards and Technology, ۲۰۰۷.
- [۵] D. C. Montgomery, E. A. Peck and G. G. Vining, Introduction to Linear Regression Analysis, ۶th ed., Hoboken: John Wiley & Sons, ۲۰۲۱.
- [۶] D. C. Montgomery and G. C. Runger, Applied Statistics and Probability for Engineers, Hoboken, NJ: John Wiley & Sons, ۲۰۱۸.
- [۷] N. Ye and Q. Chen, "An Anomaly Detection Technique Based on a Chi-Square Statistic Test," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. ۳۱, no. ۵, pp. ۳۸۷-۳۹۴, ۲۰۰۱.
- [۸] C. Sanders, Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems, No Starch Press, ۲۰۱۷.
- [۹] P. Biondi, "Scapy: Interactive packet manipulation program," ۲۰۲۰. [Online]. Available: <https://scapy.net>.
- [۱۰] A. S. Christensen, "Event-driven architecture and webhooks in modern network automation," *Journal of Network and Systems Management*, ۲۰۲۲.
- [۱۱] V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys (CSUR)*, vol. ۴۱, no. ۳, pp. ۱-۵۸, ۲۰۰۹.
- [۱۲] W. Stallings, Network Security Essentials: Applications and Standards, Pearson, ۲۰۱۷.
- [۱۳] A. Patel, "A survey of intrusion detection and prevention systems," *Information Management & Computer Security*, ۲۰۱۰.

پیوست‌ها

در این بخش، مستندات فنی و جزئیات اجرایی پروژه جهت فراهم کردن امکان بازتولید آزمایش‌ها و بررسی دقیق‌تر ساختار سیستم ارائه شده است.

پیوست الف: کدهای کامل اسکریپت‌های پایتون

در ادامه، کدهای کامل نوشته شده برای دو مرحله اصلی پروژه (آموزش مدل و مانیتورینگ زنده) آورده شده است.

۱. فایل `train_model.py` (تحلیل آفلاین و تعیین آستانه):

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import os

#Configuration - Paths based on your project structure
DATA_PATH = '../data/raw/normal_traffic.csv'
REPORT_PATH = '../reports/baseline_regression.png'

def train_anomaly_detector():
    print(f>Loading data from: {DATA_PATH}...")

    # Load the CSV file provided
    try:
        df = pd.read_csv(DATA_PATH)
    except FileNotFoundError:
        print("Error: normal_traffic.csv not found in data/raw/")
        return

    # Preprocessing: Round time to nearest second
    # This allows us to count packets per second (PPS)
    df['Time'] = df['Time'].astype(float).round(0)

    # Group by 'Time' to get the count of packets in each second
```



```

pps_data = df.groupby('Time').size().reset_index(name='Packet_Count')

# After pps_data calculation is completed:
output_path = '../data/processed/pps_data.csv'
pps_data.to_csv(output_path, index=False)
print(f"File successfully saved to: {output_path}")

# Prepare X (Time) and y (Packet Count) for Linear Regression
X = pps_data['Time'].values.reshape(1, -1)
y = pps_data['Packet_Count'].values

# Initialize and fit the Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Predict values to calculate residuals
y_pred = model.predict(X)

# Calculate Standard Error of the Estimate (Se)
#  $Se = \sqrt{\frac{\sum (y_{actual} - y_{pred})^2}{(n - 1)}}$ 
residuals = y - y_pred
se = np.sqrt(np.sum(residuals**2) / (len(y) - 1))

# Calculate Threshold ( $\sigma$ -sigma rule:  $\sigma * Se$  above the regression line)
# We use this as our anomaly detection boundary
avg_pps = np.mean(y)
threshold = avg_pps + (1 * se)

print("\n" + "="*50)
print(" REGRESSION ANALYSIS RESULTS")
print(50*"=")
print(f"Average PPS: {avg_pps:.2f}")
print(f"Standard Error (Se): {se:.4f}")
print(f"Calculated Threshold: {threshold:.2f}")
print("="*50 + "\n")

```

```
# Visualization
plt.figure(figsize=(12, 6))
plt.scatter(X, y, color='dodgerblue', alpha=0.6, label='Normal Traffic (PPS)')
plt.plot(X, y_pred, color='red', linewidth=2, label='Linear Regression Trend')
plt.axhline(y=threshold, color='green', linestyle='--', label=f'Anomaly Threshold
({threshold:.2f})')
```

```
plt.title('Baseline Traffic Analysis - Linear Regression')
plt.xlabel('Time (Seconds)')
plt.ylabel('Packets Per Second (PPS)')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.5)
```

```
# Save the report
os.makedirs(os.path.dirname(REPORT_PATH), exist_ok=True)
plt.savefig(REPORT_PATH)
print(f'Analysis plot saved to: {REPORT_PATH}')
```

```
if __name__ == "__main__":
    train_anomaly_detector()
```

۲. فایل detect_attack.py (شنود زنده و تشخیص نفوذ):

```
import scapy.all as scapy
import collections
import subprocess
import platform
import time
import requests
from datetime import datetime
```

```
--- Configuration --- #
```

```
THRESHOLD = 144.83
```

```
INTERFACE = "eth0" # Standard network interface name
```

```
N^N_WEBHOOK_URL = "http://localhost:5678/webhook/ids-alert" # Generic
Webhook Endpoint
```

```
Global states #
```

```

packet_count = 0
ip_counter = Counter()
blocked_ips = set()

def process_packet(packet):
    global packet_count, ip_counter
    packet_count += 1
    if packet.haslayer(scapy.IP):
        src_ip = packet[scapy.IP].src
        ip_counter[src_ip] += 1

def block_ip(ip_address):
    if ip_address in blocked_ips:
        print(f'[*] IP {ip_address} is already blocked. Skipping firewall injection.')
        return False

os_type = platform.system().lower
success = False

try:
    if os_type == "windows":
        Inbound Block .\ #
        cmd_in = [
            "netsh", "advfirewall", "firewall", "add", "rule",
            f'fname=IDS_AUTO_BLOCK_IN_{ip_address}',
            "dir=in", "action=block", f'remoteip={ip_address}', "profile=any"
        ]
        Outbound Block .\ #
        cmd_out = [
            "netsh", "advfirewall", "firewall", "add", "rule"

```

```

,fname=IDS_AUTO_BLOCK_OUT_{ip_address}

"dir=out", "action=block", f"remoteip={ip_address}", "profile=any"
[

res\ = subprocess.run(cmd_in, capture_output=True, text=True)
res^ = subprocess.run(cmd_out, capture_output=True, text=True)

:if res\.returncode == 0 and res^.returncode == 0
print(f"[+] Successfully blocked ALL traffic for IP {ip_address} in
Windows Firewall.")
success = True

:"elif os_type == "linux
cmd = ["iptables", "-A", "INPUT", "-s", ip_address, "-j", "DROP"]
result = subprocess.run(cmd, capture_output=True, text=True)
:if result.returncode == 0
print(f"[+] Successfully blocked IP {ip_address} in iptables.")
success = True
:else
print(f"[-] Failed to block IP {ip_address}: {result.stderr}")

:except Exception as e
print(f"[-] Error executing firewall command: {e}")

:if success
blocked_ips.add(ip_address)

return success

:()def start_monitoring
global packet_count, ip_counter
print(f"[*] Monitoring started on {INTERFACE}...")

```

```
print(f'[*] Threshold is set to: {THRESHOLD} PPS')
```

```
:while True
```

```
    packet_count = 0
```

```
    ip_counter.clear
```

```
    # Sniff packets for 1 second
```

```
    scapy.sniff(iface=INTERFACE, prn=process_packet, timeout=1, store=0)
```

```
    current_pps = packet_count
```

```
    print(f'Current Traffic: {current_pps} PPS', end='\r')
```

```
    if current_pps > THRESHOLD
```

```
        # Capture accurate system time upon attack detection
```

```
        attack_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
        print(f'\n[!!!] ANOMALY DETECTED at [{attack_time}]! Traffic: {current_pps} PPS')
```

```
        attacker_ip = "Unknown"
```

```
        if ip_counter
```

```
            [0][attacker_ip = ip_counter.most_common(1)[0]
```

```
            print(f'! Attacker IP identified: {attacker_ip}')
```

```
        # Active response execution
```

```
        block_ip(attacker_ip)
```

```
        send_alert(current_pps, attacker_ip)
```

```
        time.sleep
```

```
def send_alert(pps, attacker_ip)
```

```

} = payload
,"method": "Linear Regression Model"
,"traffic": pps"
,"threshold": THRESHOLD"
,"status": "Attack Detected & Blocked"
,"message": "Intrusion detected by Linear Regression Model"
,"student_id": "STUDENT_ID"
,"network": "Lab Environment Network"
,"blocked_ip": attacker_ip"
"action": "Active Response - Firewall Block"
{

:try
response = requests.post(N^N_WEBHOOK_URL, json=payload)
:if response.status_code == ۲۰۰
print(f'[+] Alert sent to n^n! Traffic: {pps} PPS | Target IP: {attacker_ip}')
:else
print(f'![!] n^n Error: {response.status_code}')
:except Exception as e
print(f'[-] Connection failed: {e}')

:"__if __name__ == "__main
()start_monitoring

```

مخزن پروژه در گیت‌هاب : جهت مشاهده تاریخچه توسعه اسکریپت‌ها، دسترسی به فایل‌های داده

(Dataset) و مستندات تکمیلی، می‌توانید به مخزن این پروژه در گیت‌هاب مراجعه فرمایید:

<https://github.com/Omid-Programmer۲/Network-IDS-Regression-n8n>

پیوست ب: ساختار جریان کاری در n^n (JSON Export)

برای بازسازی جریان کاری (Workflow) در پلتفرم n^n، می‌توان از قطعه کد JSON زیر استفاده کرد. با کپی کردن این متن و انتخاب گزینه **Import from JSON** در محیط n^n، تمام گره‌ها و ارتباطات وب‌هوک و بات ایتا به طور خودکار پیکربندی می‌شوند:

```
{
  "name": "IDS Notification Workflow",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "ids-alert-webhook",
        "options": {}
      },
      "type": "n^n-nodes-base.webhook",
      "typeVersion": ۲,۱,
      "position": [
        -۹۱۲,
        ۴۸
      ],
      "id": "webhook-node-id",
      "name": "Webhook",
      "webhookId": "ids-alert-webhook"
    },
    {
      "parameters": {
        "method": "POST",
        "url":
"https://eitaayar.ir/api/bot\۲۳۴۵۶:YOUR_EITAA_BOT_TOKEN_HERE/sendMessage",
        "sendBody": true,
        "contentType": "multipart-form-data",
        "bodyParameters": {
          "parameters": [
            {
              "name": "chat_id",
              "value": "YOUR_CHAT_ID"
            }
          ]
        }
      }
    }
  ]
}
```

```

    {
      "name": "text",
      "value": "=[●] IDS ALERT: Attack Detected [●]\n-----
\nStatus: {{ $json.body.status }}\nConfidence: {{ $json.body.probability }}\nMethod:
{{ $json.body.alert_type }}\nStudent ID: {{ $json.body.student_id }}\nSupervisor: {{
$json.body.supervisor }}\n-----\nTarget OS: {{ $json.body.target_os
}}\nTimestamp: {{ $json.body.timestamp }}"
    }
  ],
  "options": {},
},
"type": "n^n-nodes-base.httpRequest",
"typeVersion": 4,
"position": [
  -740,
  48
],
"id": "httprequest-node-id",
"name": "HTTP Request"
}
],
"pinData": {},
"connections": {
  "Webhook": {
    "main": [
      [
        {
          "node": "HTTP Request",
          "type": "main",
          "index": 0
        }
      ]
    ]
  }
},
"active": false,
"settings": {
  "executionOrder": "v\"
},

```



```
"tags": []  
}
```

پیوست ج: نیازمندی‌های سیستم و مشخصات محیط آزمایشگاهی

جهت اجرای صحیح پروژه، مشخصات نرم‌افزاری و سخت‌افزاری زیر در محیط آزمایشگاهی پیاده‌سازی شده است:

زبان برنامه‌نویسی: Python نسخه ۳.۱۰ یا بالاتر.

کتابخانه‌های پایتون:

- scapy: جهت شنود و تحلیل بسته‌های شبکه.
- numpy و pandas: جهت پردازش داده‌ها و محاسبات آماری.
- scikit-learn: جهت پیاده‌سازی مدل رگرسیون خطی.
- requests: جهت برقراری ارتباط با API وب‌هوک.

زیرساخت مجازی‌سازی: VMware Workstation نسخه ۱۷.

سیستم‌عامل‌های مورد استفاده:

- سیستم مانیتورینگ و مهاجم: ویندوز ۱۰ (میزبان اسکریپت پایتون و ابزار Zenmap).
- سیستم هدف (Target): ویندوز ۷ مجازی (مستقر در VMnet^۴). شایان ذکر است که معماری اسکریپت پایتون به گونه‌ای طراحی شده که قابلیت اجرا و پایش ترافیک در سایر سیستم‌عامل‌ها (مانند توزیع‌های مختلف لینوکس) را نیز دارا می‌باشد

ابزار تولید ترافیک ناهنجار: Zenmap (نسخه ویندوزی Nmap) جهت اجرای اسکن‌های تهاجمی (Intense Scan).

پیکربندی شبکه: استفاده از سویچ مجازی VMnet^۴ جهت برقراری ارتباط ایزوله بین سیستم مهاجم و هدف و شنود ترافیک توسط Scapy.

پیوست د: فرمول‌های تکمیلی آماری

در این بخش، فرمول محاسبه خطای استاندارد برآورد (S_e) که مبنای تعیین آستانه بحرانی بوده، به تفصیل آورده شده است:

$$S_e = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - 2}}$$

که در آن :

y_i : تعداد واقعی بسته‌های مشاهده شده در هر ثانیه.

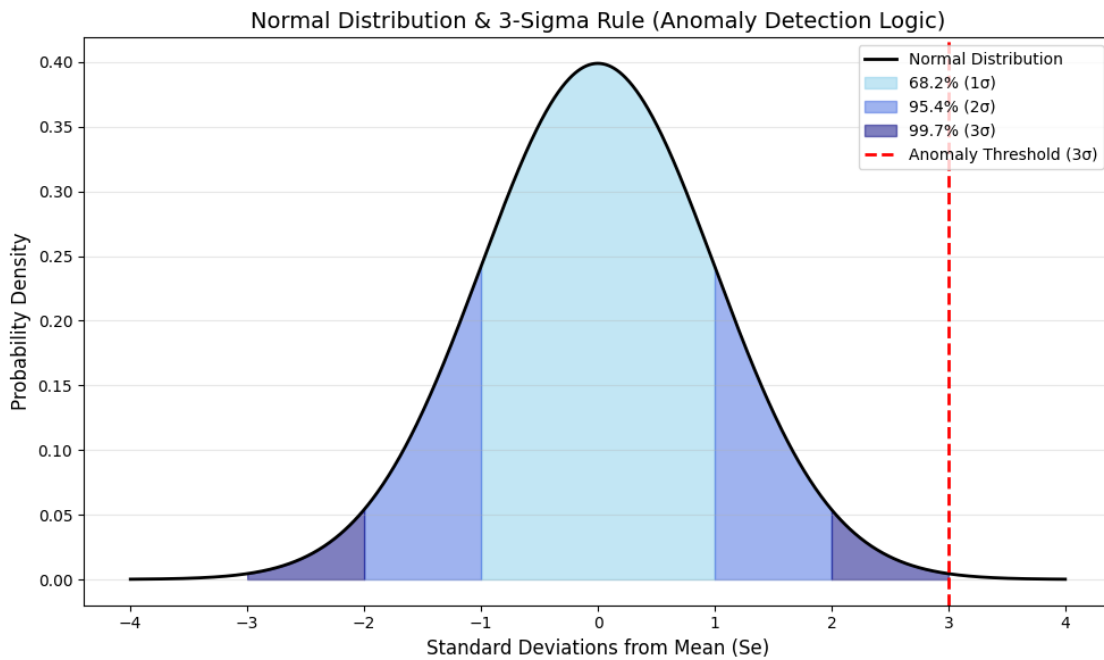
$\hat{y}_i$: تعداد بسته‌های پیش‌بینی شده توسط مدل رگرسیون.

n : تعداد کل نمونه‌های زمانی در ترافیک *Baseline*.

$n-2$: درجه آزادی برای مدل رگرسیون خطی ساده.

تحلیل آماری آستانه: آستانه $3S_e$ بر اساس قانون تجربی توزیع نرمال انتخاب شده است تا با

پوشش دادن ۹۹.۷٪ از نوسانات تصادفی ترافیک عادی، احتمال هشدار کاذب (False Positive) به کمتر از ۰.۳٪ برسد.



عکس د-۱: مدل توزیع احتمال خطا و مرز تشخیص نفوذ بر اساس قاعده ۳-سیگما